

Building a Multi-Agent Scheduling System for Fabulosa Books

A Step-by-Step Tutorial on Multi-Agent Orchestration with Next.js, Vercel AI SDK, and Neon

Part 1: Welcome to Fabulosa Books

A Bookstore with Heart

Walk into Fabulosa Books on any afternoon, and you'll know immediately why it matters. There's a particular kind of stillness here—not the kind of bookstore silence where people shuffle between shelves pretending to browse. This is a living space. The air is warm. Someone's always making coffee. A vintage armchair by the window holds a regular lost in a novel. Staff members greet people by name. There's jazz playing softly in the background, chosen with care.

[IMAGE: The Fabulosa Books storefront on a sunny afternoon in the Castro — warm light, books in the window, a handwritten “Open” sign on the door]

Fabulosa Books is an LGBTQIA+ bookstore—and that identity shapes everything about it. The shelves hold an extensive selection of queer literature: fiction, poetry, essays, memoirs, history, theory, erotica, graphic novels, children's books for queer kids and queer families. But that's just the beginning. The store also carries ephemera—stickers, postcards, zines, posters—and beyond the LGBTQIA+ titles, there's a huge general collection, too: literary fiction, art books, local history, essays on food and politics and culture. It's the kind of place where you come in looking for one book and leave with three recommendations you didn't know you needed.

Located in San Francisco's Castro neighborhood, Fabulosa Books has become a community institution. People find rare books here. They attend readings and literary events. They come to talk to staff about what they should read next, and they listen because these aren't just employees—they're readers, writers, poets, librarians, curators. They have opinions and taste and genuine passion for literature. The store is a gathering place for writers, artists, students, longtime residents, and travelers passing through who stumbled into the right bookstore by luck.

This is a small, independent business in an age of mega-chains and algorithms. Every detail matters because the team that runs it chooses to make it matter.

The Store and Its People

Alvin founded Fabulosa Books after managing the location for the previous owner. When the opportunity came to buy it, he did—because the store was worth preserving. Today, he runs it with the same care he brought to curation and customer connections, but now as the owner, making decisions about the store's future, carefully choosing the team that keeps it running.

The team is small but dedicated: eight people total, six who work regular shifts and two who come in on-call. It's the kind of crew where everyone knows the store's inventory by heart, where someone can recommend the perfect book for a customer they've never met, where shift changes feel like transitions between chapters rather than just clocking in and out.

[IMAGE: A simple diagram showing the team — 6 regular employees (Alvin, Melissa, Marcus, Dylan, Joel, Billy) and 2 on-call employees (Johnny Ray, Carly), with names and small icons]

They host readings and events. They put together staff pick displays that actually move inventory—customers trust their recommendations because their picks are personal and thoughtful, not algorithm-driven. Many of them have deep roots in San Francisco's literary and bookselling world. They're woven into the fabric of the neighborhood in ways that big bookstores can't replicate.

But managing all this—especially the scheduling—has been a mess.

The Problem

For years, Fabulosa Books managed its schedule the old-fashioned way: a handwritten schedule pinned to the breakroom wall, updated with markers and corrections, sometimes with sticky notes covering up conflicting scribbles.

[IMAGE: An illustration of a messy paper schedule taped to a wall — crossed-out names, sticky notes, arrows, coffee stains. Caption: "The current system."]

In theory, it worked. In practice? It was chaos.

Step 1: Understand why a paper schedule fails at scale.

- Employees couldn't check the schedule from home. They had to come in or call the store.
- Last-minute changes were a nightmare. An erased shift and a hastily scrawled note caused confusion and frustration.
- Swaps and requests were informal—a conversation in the break room, a text message, or a note left on the schedule itself.
- Nobody knew weeks in advance who was working what. Planning personal time was guesswork.
- Management spent an embarrassing amount of time just answering "When am I working?" questions.
- If the schedule got damaged or lost, there was no backup. It was pure anxiety.

But the biggest problem wasn't access or logistics—it was the *thinking*. Every week, Alvin sat down and tried to solve a puzzle with too many pieces: Who's available? Who worked closing last week? Is Marcus already at five shifts? Can we cover Saturday with one on-call staff, or do we need two? Is this fair to everyone?

That mental load—the reasoning, the balancing, the judgment—is what takes the most time. And it's exactly the kind of work that AI agents are built to handle.

What We'll Build

This is where things get interesting. We're not just going to build a digital schedule—we're going to build a *scheduling assistant* powered by multiple AI agents that work together.

Step 2: Understand the vision.

Billy, the newest member of the team and a struggling writer with a passion for literary fiction, volunteered to build this. But Billy didn't want to just replace the paper schedule with a grid on a screen. He wanted to replace the *thinking*—the hours Alvin spends every week balancing availability, fairness, coverage, and special requests.

His idea: build a system where you can say “Schedule next week” and multiple specialized AI agents coordinate to produce a fair, complete schedule. One agent checks who's available. Another ensures the shifts are distributed fairly. A third validates that the store is properly covered. And a coordinator agent orchestrates the whole thing—like Alvin delegating to his team, but in code.

[IMAGE: A high-level architecture diagram showing: User → Chat Interface → Coordinator Agent → three specialist agents (Availability, Fairness, Coverage) → Database. Arrows show the flow of requests and responses between them.]

The Stack:

- **Frontend:** A responsive web interface built with React and Next.js
- **AI Layer:** Vercel AI SDK for building and orchestrating agents
- **AI Models:** Vercel AI Gateway for model access (Anthropic, OpenAI, Google — one API key)
- **Database:** Neon (serverless Postgres) to store schedule data
- **Hosting:** Vercel for zero-friction deployments

The Schedule:

The store hours are fixed: 10am–8pm daily (10am–9pm on Saturdays). Shifts are divided into three slots per day:

- **Opening Shift:** 10am–4pm
- **Mid Shift:** 12pm–6pm
- **Closing Shift:** 2pm–8pm (or 2pm–9pm Saturdays)

Each shift gets assigned to one employee. The AI agents handle the reasoning—who should work when, whether it's fair, whether there are gaps. The manager can review, adjust, and approve. And the whole thing is accessible from any device.

The Agents:

Agent	Role	Analogy
Availability Agent	Checks who's free and when	“Let me check everyone's calendar”

Agent	Role	Analogy
Fairness Agent	Ensures equitable shift distribution	“Marcus has had closing three weeks in a row—let’s fix that”
Coverage Agent	Validates store staffing rules	“We need three people every day, and Saturday closing is an extra hour”
Schedule Coordinator	Orchestrates the other three	Alvin delegating to his team

By the end of this tutorial, you’ll have a fully functional multi-agent scheduling system that the team at Fabulosa Books can use—and you’ll understand how to design, build, and orchestrate AI agents for any problem.

Ready to dive in? Let’s go.

Part 2: How Agents Think

From Paper to Web App (The Quick Version)

Before we talk about agents, let’s cover the basics: what is a web app?

You’ve used web apps without thinking about it. Google Docs. Gmail. Your online banking. These aren’t regular websites—they’re applications that run in your browser. You can click buttons, type things, save information, and see changes happen instantly. The key difference: a regular website shows you information. A web app lets you *do* things.

That’s what we’re building for Fabulosa Books: something everyone can access from their phone or laptop, no installation required. But unlike a typical web app where all the logic is hand-coded, ours will have AI agents that *reason* about the schedule.

[IMAGE: A simple before/after comparison — left side shows a static web page (read-only), right side shows a web app with interactive elements and an AI chat panel. Caption: “Web apps do things. AI-powered web apps think about what to do.”]

What Is an AI Agent?

Step 3: Understand the difference between a chatbot and an agent.

You’ve probably used ChatGPT or a similar chatbot. You type a question, you get an answer. That’s useful, but it’s reactive—the AI waits for you to ask and then responds. It doesn’t *do* anything in the world.

An agent is different. An agent has a *goal* and *tools*. When you give it a task, it doesn’t just respond—it figures out what steps to take, uses its tools to gather information or make changes, evaluates the results, and keeps going until the task is done.

Think of it this way:

- **Chatbot:** You ask “Who’s available Thursday?” and it says “I don’t have access to your schedule.”
- **Agent:** You ask “Who’s available Thursday?” and it queries the database, checks existing assignments, cross-references time-off requests, and tells you “Dylan, Joel, and Carly are available. Marcus and Billy are already scheduled.”

The difference is the tools. An agent without tools is just a chatbot. An agent *with* tools can look things up, validate data, and take actions.

[IMAGE: A diagram comparing a chatbot (single arrow: question → answer) vs. an agent (loop: goal → think → use tool → observe result → think again → respond). The agent’s loop should show 2-3 iterations before responding.]

```
Chatbot:
  User: "Who's free Thursday?"
  AI: "I'd need to check your schedule system."

Agent:
  User: "Who's free Thursday?"
  AI thinks: "I need to check the schedule database."
  AI calls tool: getSchedule(thursday)
  AI observes: Marcus=opening, Billy=mid, unassigned=closing
  AI thinks: "Now I need the full employee list to find who's NOT scheduled."
  AI calls tool: getEmployees()
  AI observes: [Alvin, Melissa, Marcus, Dylan, Joel, Billy, Johnny Ray, Carly]
  AI responds: "Dylan, Joel, Alvin, Melissa, Johnny Ray, and Carly are
               available Thursday. Marcus has opening and Billy has mid."
```

Why Multiple Agents?

Step 4: Understand why one agent isn’t enough.

You might wonder: why not build one super-agent that does everything? The same reason Alvin doesn’t run the entire bookstore alone.

One agent trying to handle availability, fairness, coverage rules, notifications, and schedule generation all at once would be like asking one employee to staff the register, shelf returns, answer the phone, make coffee, host an event, AND figure out next week’s schedule—simultaneously. The results would be mediocre at best.

Specialization works. When you split responsibilities across multiple agents, each one can be:

- **Focused:** It has a clear, narrow job. The Fairness Agent doesn’t worry about store coverage—it only thinks about whether shifts are distributed equitably.
- **Expert:** Its instructions and tools are tailored to its domain. The Coverage Agent knows that Saturday closing is 2pm–9pm, not 2pm–8pm. The Availability Agent knows how to query the database efficiently.
- **Testable:** You can verify each agent independently. Does the Fairness Agent correctly flag when someone has three closing shifts in a row? You can test that without involving the other agents.
- **Improvable:** You can upgrade one agent without touching the others. Want better fairness logic? Update the Fairness Agent’s instructions and tools. Everything else stays the same.

[IMAGE: Two diagrams side by side. Left: a single “Super Agent” with a tangled web of responsibilities and tools, looking overwhelmed. Right: three specialized agents, each with 2-3 focused tools, connected by clean arrows to a coordinator. Caption: “Specialization beats monolithic design.”]

The Orchestrator Pattern

Step 5: Understand how agents coordinate.

If you have multiple specialized agents, someone has to coordinate them. That’s the orchestrator.

Think about how Alvin manages the bookstore. He doesn’t shelve books, run the register, AND decide the schedule simultaneously. He *delegates*. He might say: “Melissa, can you check who’s available next week? Marcus, does the event schedule conflict with anything? Joel, make sure we have coverage for Saturday.” Then he takes their answers and makes the final decision.

That’s exactly what our Schedule Coordinator agent does:

1. It receives a request from the user: “Schedule next week”
2. It breaks the request into subtasks
3. It delegates each subtask to the right specialist agent
4. It collects the results
5. It synthesizes a final answer

In the Vercel AI SDK, this pattern is implemented naturally: the coordinator is an agent whose *tools* are the other agents. When it decides to check availability, it calls the Availability Agent as a tool. When it needs to validate fairness, it calls the Fairness Agent. The framework handles the rest.

[IMAGE: A flowchart showing the orchestrator pattern for a “Schedule next week” request: 1. User sends “Schedule next week” 2. Coordinator receives the request 3. Coordinator calls Availability Agent → gets who’s free each day 4. Coordinator calls Fairness Agent → gets shift distribution analysis 5. Coordinator calls Coverage Agent → validates proposed schedule 6. Coordinator synthesizes results → returns complete schedule to user Each step should be a distinct box with arrows between them.]

```
User: "Schedule next week"

Coordinator thinks: "I need to build a schedule. Let me check availability first."
  → calls Availability Agent
  ← "Here's who's available each day: ..."

Coordinator thinks: "Now let me check if the current distribution is fair."
  → calls Fairness Agent
  ← "Marcus has had 3 closing shifts in a row. Dylan hasn't worked Saturday in a month."

Coordinator thinks: "Let me draft a schedule accounting for fairness, then validate coverage."
  → calls Coverage Agent with proposed schedule
  ← "All shifts covered. Saturday closing correctly set to 2pm-9pm."

Coordinator: "Here's the proposed schedule for next week: ..."
```

Tools: How Agents Act on the World

Step 6: Understand what tools are.

A tool is a specific capability you give to an agent. Without tools, an agent can only talk. With tools, it can *do things*.

In the Vercel AI SDK, a tool has three parts:

1. **A description** — what the tool does (in plain English, so the AI understands when to use it)
2. **An input schema** — what information the tool needs (defined with Zod, a TypeScript validation library)
3. **An execute function** — what happens when the tool is called (your actual code)

Here's a preview of what a real tool looks like:

```
const getEmployees = tool({
  description: 'Get all employees at Fabulosa Books with their role (regular or on-call)',
  inputSchema: z.object({}),
  execute: async () => {
    const employees = await getAllEmployees()
    return employees
  },
})
```

The AI model never sees your code. It sees the description and the input schema. Based on those, it decides *when* to call the tool and *what inputs to provide*. Your execute function does the actual work.

[IMAGE: An annotated code snippet of a tool definition, with arrows pointing to each part: description (“What it does — the AI reads this”), inputSchema (“What it needs — the AI fills this in”), execute (“What happens — your code runs this”). Caption: “Anatomy of an AI SDK tool.”]

Our Stack

Step 7: Know the tools we'll use to build this.

Tool	What It Does	Why We Need It
Next.js	Full-stack web framework	Builds the UI and API routes in one project
Vercel AI SDK	Agent and tool framework	Creates agents, manages tool loops, streams responses
Vercel AI Gateway	Model access API	One API key to use Claude, GPT, Gemini, and more
Neon	Serverless Postgres database	Stores employees, schedules, and shift history
Vercel	Hosting platform	Deploys the app with zero configuration
Zod	Schema validation	Defines tool input/output shapes with

Tool	What It Does	Why We Need It
		type safety

[IMAGE: A tech stack diagram showing how the pieces connect: Browser → Next.js (React UI + API Routes) → Vercel AI SDK (agents + tools) → AI Gateway (models) and Neon (data). Vercel wraps the whole thing as the hosting layer.]

These tools work together naturally. Next.js provides the web framework. The AI SDK provides the agent primitives. The AI Gateway provides model access without managing individual provider accounts. Neon stores the data. Vercel hosts it all. You write TypeScript everywhere.

Let's set it up.

Part 3: Setting Up Shop

Before We Write a Single Line of Code

Building a web app is a bit like opening a bookstore. Before you arrange the shelves or order inventory, you need a space. You need keys. You need a plan for how things will be organized.

In web development, that space is your “development environment”—the set of tools and accounts that let you write code, test it, and eventually put it online for everyone to see. This part walks you through setting all of that up.

What You'll Need

Step 8: Install Node.js.

Node.js is the engine that runs JavaScript outside of a web browser. When we write our Next.js app, Node.js is what actually runs it on your machine while you're building and testing.

Go to nodejs.org and download the version labeled “LTS” (Long Term Support). Install it like any other program. Then open your terminal and verify:

```
node --version
```

[IMAGE: Screenshot of a terminal showing `node --version` with output like `v22.x.x`.
Caption: “If you see a version number, you're good.”]

Step 9: Install a code editor.

Download Visual Studio Code from code.visualstudio.com. It's free, and it's what most developers use.

[IMAGE: Screenshot of VS Code's download page with the download button highlighted]

Step 10: Create a GitHub account.

Sign up at github.com. GitHub is like a cloud backup for code, but smarter—it tracks every change you make, so you can always go back if something breaks.

[IMAGE: Screenshot of the GitHub sign-up page]

Git: Your Safety Net

Step 11: Understand what Git does.

Git tracks changes to your files. Every time you make a meaningful change to your code, you tell Git, “Hey, save this version.” Git remembers. If you break something tomorrow, you can go back to today’s version.

GitHub is where your Git history lives online. Your computer has a copy. GitHub has a copy. They stay in sync. And when we connect GitHub to Vercel later, every time you push new code to GitHub, Vercel will automatically update your live website.

[IMAGE: A diagram showing the Git workflow: Local Files → git commit → Local Repository → git push → GitHub → Vercel auto-deploys. Arrows flow left to right.]

Creating the Project

Step 12: Create the Next.js project.

Open your terminal and run:

```
npx create-next-app@latest fabulosa-scheduler
```

When prompted, answer:

- **TypeScript?** Yes
- **ESLint?** Yes
- **Tailwind CSS?** Yes
- **src/ directory?** No
- **App Router?** Yes
- **Customize import alias?** No

[IMAGE: Screenshot of the terminal showing the create-next-app prompts with the recommended answers selected]

Step 13: Enter the project and start the dev server.

```
cd fabulosa-scheduler  
npm run dev
```

Open your browser and go to `http://localhost:3000`. You should see the Next.js welcome page.

[IMAGE: Screenshot of the browser showing the default Next.js welcome page at localhost:3000]

Step 14: Install the AI SDK and Zod.

Stop the dev server (Ctrl+C) and install the packages we’ll need for agents:

```
npm install ai @ai-sdk/react zod
```

- `ai` — the core Vercel AI SDK (agents, tools, streaming)
- `@ai-sdk/react` — React hooks for chat interfaces (`useChat`)
- `zod` — schema validation for tool inputs

[IMAGE: Screenshot of the terminal showing the npm install command completing successfully, with the three packages listed]

Step 15: Install the Neon serverless driver.

```
npm install @neondatabase/serverless
```

For local development, we'll also use a lightweight SQLite stand-in:

```
npm install sql.js
npm install --save-dev @types/sql-js
```

[IMAGE: Screenshot of the terminal showing the Neon and sql.js packages installed]

Putting It on GitHub

Step 16: Initialize Git and push to GitHub.

```
git init
git add -A
git commit -m "Initial commit: Fabulosa Books multi-agent scheduler"
```

Go to github.com, click “+”, choose “New repository”, name it `fabulosa-scheduler`, and create it. Then run:

```
git remote add origin https://github.com/YOUR-USERNAME/fabulosa-scheduler.git
git branch -M main
git push -u origin main
```

[IMAGE: Screenshot of the GitHub repository page after the first push, showing the project files]

Connecting to Vercel

Step 17: Deploy to Vercel.

Go to vercel.com and sign up with your GitHub account. Click “Add New Project,” find `fabulosa-scheduler`, and click “Import.” Vercel detects it’s a Next.js project automatically. Click “Deploy.”

[IMAGE: Screenshot of the Vercel dashboard showing the import screen with `fabulosa-scheduler` selected]

Wait about a minute. When it’s done, you’ll see a celebration screen with a link to your live site.

[IMAGE: Screenshot of the Vercel deployment success page with confetti and the live URL]

Adding the Database

Step 18: Create a Neon database through Vercel.

From your Vercel project dashboard, go to the “Storage” tab. Click “Create Database” and choose Neon. Create a free Postgres database and connect it to all three environments (Development, Preview, Production).

Vercel creates an environment variable called `DATABASE_URL` that connects your app to the database automatically.

[IMAGE: Screenshot of the Vercel Storage tab showing the Neon database creation dialog with all three environments selected]

Setting Up the AI Gateway

Step 19: Get your AI Gateway API key.

Go to your Vercel dashboard → AI Gateway → API Keys. Create a new key.

[IMAGE: Screenshot of the Vercel AI Gateway API Keys page with the “Create Key” button highlighted]

Step 20: Add the API key to your project.

In your Vercel project settings, go to Environment Variables and add:

```
AI_GATEWAY_API_KEY=your_key_here
```

For local development, pull the environment variables:

```
vercel env pull .env.local
```

[IMAGE: Screenshot of the terminal showing `vercel env pull` completing, with a `.env.local` file created]

What We’ve Done

Step 21: Verify your setup.

You now have:

- A Next.js project with the AI SDK, Zod, and Neon driver installed
- A GitHub repository backing up every change
- A Vercel deployment that auto-publishes on every push
- A Neon database ready to store schedule data
- An AI Gateway API key for model access

[IMAGE: A checklist diagram showing all five items above with green checkmarks. Caption: “A complete development pipeline. Companies with dedicated engineering teams use this exact setup.”]

The scaffolding is up. Now let’s build the foundation.

Part 4: The Foundation — Data and Database

The Plan

Before we build agents, we need the foundation they'll work with: the data model, the database, the API, and the basic schedule UI. This is the “non-AI” part of the app—the solid ground the agents will stand on.

Defining the Data

Step 22: Create the TypeScript types.

Create a `types` folder in your project root with an `index.ts` file:

```
// types/index.ts

export interface Employee {
  id: number
  name: string
  is_on_call: number
}

export interface ScheduleEntry {
  id: number
  day_of_week: number
  shift_type: string
  employee_name: string | null
  week_start_date: string
}

export interface ShiftSlot {
  dayOfWeek: number
  shiftType: string
  employeeName: string | null
}
```

An `Employee` has an id, a name, and a flag for whether they're on-call. A `ScheduleEntry` records who's working which shift on which day of which week. `string | null` means the employee name can be empty—because sometimes a shift hasn't been assigned yet.

[IMAGE: A diagram showing the relationship between the three types — Employee feeds into ScheduleEntry (via employee_name), and ShiftSlot is a simplified view of a ScheduleEntry for the UI]

The Utility Belt

Step 23: Create helper functions.

Create `lib/utils.ts`:

```
// lib/utils.ts

export const DAYS_OF_WEEK = [
  'Sunday', 'Monday', 'Tuesday', 'Wednesday',
  'Thursday', 'Friday', 'Saturday'
]

export const SHIFTS = [
```

```

    { type: 'opening', label: 'Opening', start: '10am', end: '4pm' },
    { type: 'mid', label: 'Mid', start: '12pm', end: '6pm' },
    { type: 'closing', label: 'Closing', start: '2pm', end: '8pm' },
  ]

export const SHIFTS_SATURDAY = [
  { type: 'opening', label: 'Opening', start: '10am', end: '4pm' },
  { type: 'mid', label: 'Mid', start: '12pm', end: '6pm' },
  { type: 'closing', label: 'Closing', start: '2pm', end: '9pm' },
]

export function getShiftsForDay(dayOfWeek: number) {
  return dayOfWeek === 6 ? SHIFTS_SATURDAY : SHIFTS
}

export function getWeekStartDate(date?: Date): string {
  const d = date ? new Date(date) : new Date()
  const day = d.getDay()
  const diff = d.getDate() - day
  const weekStart = new Date(d.setDate(diff))
  return weekStart.toISOString().split('T')[0]
}

export function getDateFromDayOfWeek(
  weekStart: string,
  dayOfWeek: number
): Date {
  const date = new Date(weekStart)
  date.setDate(date.getDate() + dayOfWeek)
  return date
}

export function formatDate(date: Date): string {
  return date.toLocaleDateString('en-US', {
    month: 'short',
    day: 'numeric',
  })
}

```

Nothing dramatic—just useful tools. `getWeekStartDate` figures out what Sunday starts the current week. `getShiftsForDay` returns the right shift times, accounting for Saturday’s later closing.

[IMAGE: A small reference card showing the three daily shifts with their times, and Saturday’s closing shift highlighted as different (9pm vs 8pm)]

The Database Layer

Step 24: Build the database module.

This is the longest file in the project, but it’s straightforward. Create `lib/db.ts`. It has two halves: one for Neon (production) and one for local SQLite (development). The app automatically uses the right one based on whether `DATABASE_URL` exists.

```

// lib/db.ts

import { neon } from '@neondatabase/serverless'

// ---- Neon (production) helpers ----

function getSQL() {

```

```

const url = process.env.DATABASE_URL
if (!url) throw new Error('DATABASE_URL is not set')
return neon(url)
}

async function initializeNeonSchema() {
  const sql = getSQL()

  await sql`
    CREATE TABLE IF NOT EXISTS employees (
      id SERIAL PRIMARY KEY,
      name TEXT NOT NULL UNIQUE,
      is_on_call BOOLEAN DEFAULT false,
      created_at TIMESTAMPTZ DEFAULT NOW()
    )
  `

  await sql`
    CREATE TABLE IF NOT EXISTS schedule (
      id SERIAL PRIMARY KEY,
      day_of_week INTEGER NOT NULL,
      shift_type TEXT NOT NULL,
      employee_name TEXT,
      week_start_date TEXT NOT NULL,
      created_at TIMESTAMPTZ DEFAULT NOW(),
      updated_at TIMESTAMPTZ DEFAULT NOW(),
      UNIQUE(week_start_date, day_of_week, shift_type)
    )
  `

  // Seed employees if table is empty
  const existing = await sql`SELECT count(*) as cnt FROM employees`
  if (Number(existing[0].cnt) === 0) {
    const employees = [
      { name: 'Alvin', isOnCall: false },
      { name: 'Melissa', isOnCall: false },
      { name: 'Marcus', isOnCall: false },
      { name: 'Dylan', isOnCall: false },
      { name: 'Joel', isOnCall: false },
      { name: 'Billy', isOnCall: false },
      { name: 'Johnny Ray', isOnCall: true },
      { name: 'Carly', isOnCall: true },
    ]
    for (const emp of employees) {
      await sql`
        INSERT INTO employees (name, is_on_call)
        VALUES (${emp.name}, ${emp.isOnCall})
        ON CONFLICT (name) DO NOTHING
      `
    }
  }
}

let schemaInitialized = false

async function ensureNeonSchema() {
  if (!schemaInitialized) {
    await initializeNeonSchema()
    schemaInitialized = true
  }
}

```

That **UNIQUE** constraint on the schedule table is important—it means you can’t accidentally assign two people to the same shift. The database itself enforces the rules.

Step 25: Add the local development database.

Below the Neon code in `lib/db.ts`, add the SQLite fallback for local development:

```
// ---- sql.js (local dev) helpers ----

import initSqlJs, { Database as SqlJsDatabase } from 'sql.js'
import fs from 'fs'
import path from 'path'

let localDb: SqlJsDatabase | null = null

const getDbPath = () => {
  const dbDir = path.join(process.cwd(), 'data')
  const dbFile = path.join(dbDir, 'schedule.db.json')
  return { dbDir, dbFile }
}

function saveLocalDb() {
  if (!localDb) return
  const { dbDir, dbFile } = getDbPath()
  if (!fs.existsSync(dbDir)) {
    fs.mkdirSync(dbDir, { recursive: true })
  }
  const data = localDb.export()
  const buffer = Buffer.from(data)
  fs.writeFileSync(dbFile, JSON.stringify(Array.from(buffer)))
}

function initializeLocalSchema() {
  if (!localDb) return

  localDb.run(`
    CREATE TABLE IF NOT EXISTS employees (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL UNIQUE,
      is_on_call BOOLEAN DEFAULT 0,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )
  `)

  localDb.run(`
    CREATE TABLE IF NOT EXISTS schedule (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      day_of_week INTEGER NOT NULL,
      shift_type TEXT NOT NULL,
      employee_name TEXT,
      week_start_date TEXT NOT NULL,
      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
      UNIQUE(week_start_date, day_of_week, shift_type)
    )
  `)

  const employees = [
    { name: 'Alvin', isOnCall: false },
    { name: 'Melissa', isOnCall: false },
    { name: 'Marcus', isOnCall: false },
    { name: 'Dylan', isOnCall: false },
    { name: 'Joel', isOnCall: false },
    { name: 'Billy', isOnCall: false },
    { name: 'Johnny Ray', isOnCall: true },
    { name: 'Carly', isOnCall: true },
  ]
}
```

```

employees.forEach((emp) => {
  try {
    localDb!.run(
      'INSERT INTO employees (name, is_on_call) VALUES (?, ?)',
      [emp.name, emp.isOnCall ? 1 : 0]
    )
  } catch {
    // already exists
  }
})

saveLocalDb()
}

async function getLocalDb(): Promise<SqlJsDatabase> {
  if (localDb) return localDb

  const SQL = await initSqlJs()
  const { dbDir, dbFile } = getDbPath()

  if (!fs.existsSync(dbDir)) {
    fs.mkdirSync(dbDir, { recursive: true })
  }

  if (fs.existsSync(dbFile)) {
    try {
      const content = fs.readFileSync(dbFile, 'utf-8')
      const data = JSON.parse(content)
      localDb = new SQL.Database(new Uint8Array(data))
    } catch {
      localDb = new SQL.Database()
      initializeLocalSchema()
    }
  } else {
    localDb = new SQL.Database()
    initializeLocalSchema()
  }

  return localDb!
}

```

[IMAGE: A diagram showing the dual-database architecture: Development (your laptop) → sql.js (SQLite) and Production (Vercel) → Neon (Postgres). Both expose the same API. Caption: “Same code, different database. The app doesn’t know or care which one is running.”]

Step 26: Add the public API functions.

These are the functions the rest of the app (and our agents) will use:

```

// ---- Detect which backend to use ----

const useNeon = !!process.env.DATABASE_URL

// ---- Public API ----

export async function getAllEmployees() {
  if (useNeon) {
    await ensureNeonSchema()
    const sql = getSQL()
    const rows = await sql`SELECT id, name, is_on_call FROM employees ORDER BY name`
    return rows.map((r: any) => ({
      id: r.id,

```

```

        name: r.name,
        is_on_call: r.is_on_call,
      )))
    } else {
      const db = await getLocalDb()
      const result = db.exec(
        'SELECT id, name, is_on_call FROM employees ORDER BY name'
      )
      if (result.length === 0) return []
      return result[0].values.map((row: any) => ({
        id: row[0],
        name: row[1],
        is_on_call: row[2],
      })))
    }
  }
}

export async function getWeekSchedule(weekStartDate: string) {
  if (useNeon) {
    await ensureNeonSchema()
    const sql = getSQL()
    const rows = await sql`
      SELECT id, day_of_week, shift_type, employee_name, week_start_date
      FROM schedule
      WHERE week_start_date = ${weekStartDate}
      ORDER BY day_of_week, shift_type
    `
    return rows.map((r: any) => ({
      id: r.id,
      day_of_week: r.day_of_week,
      shift_type: r.shift_type,
      employee_name: r.employee_name,
      week_start_date: r.week_start_date,
    })))
  } else {
    const db = await getLocalDb()
    const result = db.exec(
      'SELECT id, day_of_week, shift_type, employee_name, week_start_date FROM schedule
      WHERE week_start_date = ? ORDER BY day_of_week, shift_type',
      [weekStartDate]
    )
    if (result.length === 0) return []
    return result[0].values.map((row: any) => ({
      id: row[0],
      day_of_week: row[1],
      shift_type: row[2],
      employee_name: row[3],
      week_start_date: row[4],
    })))
  }
}

export async function updateSchedule(
  weekStartDate: string,
  dayOfWeek: number,
  shiftType: string,
  employeeName: string | null
) {
  if (useNeon) {
    await ensureNeonSchema()
    const sql = getSQL()

    if (!employeeName) {
      await sql`
        DELETE FROM schedule
      `
    }
  }
}

```

```

        WHERE week_start_date = ${weekStartDate}
        AND day_of_week = ${dayOfWeek}
        AND shift_type = ${shiftType}
    },
    } else {
        await sql`
            INSERT INTO schedule (week_start_date, day_of_week, shift_type, employee_name)
            VALUES (${weekStartDate}, ${dayOfWeek}, ${shiftType}, ${employeeName})
            ON CONFLICT (week_start_date, day_of_week, shift_type)
            DO UPDATE SET employee_name = ${employeeName}, updated_at = NOW()
        `
    }
    } else {
        const db = await getLocalDb()

        if (!employeeName) {
            db.run(
                'DELETE FROM schedule WHERE week_start_date = ? AND day_of_week = ? AND shift_type = ?',
                [weekStartDate, dayOfWeek, shiftType]
            )
        } else {
            const check = db.exec(
                'SELECT id FROM schedule WHERE week_start_date = ? AND day_of_week = ? AND shift_type = ?',
                [weekStartDate, dayOfWeek, shiftType]
            )

            if (check.length > 0 && check[0].values.length > 0) {
                db.run(
                    'UPDATE schedule SET employee_name = ?, updated_at = CURRENT_TIMESTAMP WHERE week_start_date = ? AND day_of_week = ? AND shift_type = ?',
                    [employeeName, weekStartDate, dayOfWeek, shiftType]
                )
            } else {
                db.run(
                    'INSERT INTO schedule (week_start_date, day_of_week, shift_type, employee_name) VALUES (?, ?, ?, ?)',
                    [weekStartDate, dayOfWeek, shiftType, employeeName]
                )
            }
        }
    }

    saveLocalDb()
}

```

The API Routes

Step 27: Create the employees API route.

Create `app/api/employees/route.ts`:

```

// app/api/employees/route.ts

import { getAllEmployees } from '@/lib/db'
import { NextResponse } from 'next/server'

export const dynamic = 'force-dynamic'

export async function GET() {
    try {
        const employees = await getAllEmployees()
        return NextResponse.json(

```

```

        employees.map((emp: any) => ({
            ...emp,
            is_on_call: Boolean(emp.is_on_call),
        })))
    )
} catch (error) {
    console.error('Error fetching employees:', error)
    return NextResponse.json(
        { error: 'Failed to fetch employees' },
        { status: 500 }
    )
}
}

```

Step 28: Create the schedule API route.

Create `app/api/schedule/route.ts`:

```

// app/api/schedule/route.ts

import { NextRequest, NextResponse } from 'next/server'
import { getWeekSchedule, updateSchedule } from '@lib/db'
import { getWeekStartDate } from '@lib/utils'

export const dynamic = 'force-dynamic'

export async function GET(request: NextRequest) {
    try {
        const searchParams = request.nextUrl.searchParams
        const weekStart = searchParams.get('weekStart') || getWeekStartDate()

        const schedule = await getWeekSchedule(weekStart)
        return NextResponse.json({ schedule, weekStart })
    } catch (error) {
        console.error('Error fetching schedule:', error)
        return NextResponse.json(
            { error: 'Failed to fetch schedule' },
            { status: 500 }
        )
    }
}

export async function PUT(request: NextRequest) {
    try {
        const body = await request.json()
        const { weekStart, dayOfWeek, shiftType, employeeName } = body

        if (
            weekStart === undefined ||
            dayOfWeek === undefined ||
            shiftType === undefined
        ) {
            return NextResponse.json(
                { error: 'Missing required fields' },
                { status: 400 }
            )
        }

        await updateSchedule(weekStart, dayOfWeek, shiftType, employeeName || null)

        return NextResponse.json({ success: true })
    } catch (error) {
        console.error('Error updating schedule:', error)
        return NextResponse.json(

```

```

    { error: 'Failed to update schedule' },
    { status: 500 }
  )
}
}

```

[IMAGE: A diagram showing the API architecture: Browser makes GET/PUT requests → API Routes → Database Layer → Neon/SQLite. Caption: “The browser never talks to the database directly. API routes are the middleman.”]

The Schedule UI

Step 29: Build the ShiftCell component.

Create `components/ShiftCell.tsx` — each cell in the schedule grid is its own component. When you click it, a dropdown shows all employees:

```

// components/ShiftCell.tsx
'use client'

import { Employee } from '@types'

interface ShiftCellProps {
  dayOfWeek: number
  shiftType: string
  currentEmployee: string | null
  employees: Employee[]
  onUpdate: (dayOfWeek: number, shiftType: string, employeeName: string | null) => void
  isSaturday: boolean
}

export default function ShiftCell({
  dayOfWeek,
  shiftType,
  currentEmployee,
  employees,
  onUpdate,
  isSaturday,
}: ShiftCellProps) {
  const currentEmployeeData = employees.find(e => e.name === currentEmployee)

  const bgColor = currentEmployee
    ? currentEmployeeData?.is_on_call
      ? 'bg-amber-700 text-white'
      : 'bg-amber-500 text-white'
    : 'bg-amber-50 text-stone-600'

  const regularEmployees = employees.filter(e => !e.is_on_call)
  const onCallEmployees = employees.filter(e => e.is_on_call)

  return (
    <select
      value={currentEmployee || ''}
      onChange={(e) => onUpdate(dayOfWeek, shiftType, e.target.value || null)}
      className={`w-full p-2 rounded text-sm font-medium cursor-pointer ${bgColor}`}
    >
      <option value="">Unassigned</option>
      <optgroup label="Regular">
        {regularEmployees.map(emp => (
          <option key={emp.id} value={emp.name}>{emp.name}</option>
        ))}
      </optgroup>
    </select>
  )
}

```

```

        <optgroup label="On-Call">
          {onCallEmployees.map(emp => (
            <option key={emp.id} value={emp.name}>{emp.name}</option>
          ))}
        </optgroup>
      </select>
    )
  }
}

```

Step 30: Build the ScheduleGrid component.

Create `components/ScheduleGrid.tsx`:

```

// components/ScheduleGrid.tsx
'use client'

import { useState, useEffect, useCallback } from 'react'
import { Employee, ScheduleEntry } from '@types'
import { DAYS_OF_WEEK, getWeekStartDate, formatDate, getDateFromDayOfWeek } from '@lib/Utils'
import ShiftCell from '../ShiftCell'

export default function ScheduleGrid() {
  const [weekStart, setWeekStart] = useState(getWeekStartDate())
  const [schedule, setSchedule] = useState<ScheduleEntry[]>([])
  const [employees, setEmployees] = useState<Employee[]>([])

  const fetchData = useCallback(async () => {
    const [empRes, schedRes] = await Promise.all([
      fetch('/api/employees'),
      fetch(`/api/schedule?weekStart=${weekStart}`),
    ])
    const empData = await empRes.json()
    const schedData = await schedRes.json()
    setEmployees(empData)
    setSchedule(schedData.schedule)
  }, [weekStart])

  useEffect(() => { fetchData() }, [fetchData])

  const handleShiftUpdate = async (
    dayOfWeek: number,
    shiftType: string,
    employeeName: string | null
  ) => {
    await fetch('/api/schedule', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ weekStart, dayOfWeek, shiftType, employeeName }),
    })
    fetchData()
  }

  const getEmployeeForShift = (dayOfWeek: number, shiftType: string) => {
    const entry = schedule.find(
      s => s.day_of_week === dayOfWeek && s.shift_type === shiftType
    )
    return entry?.employee_name || null
  }

  const navigateWeek = (offset: number) => {
    const current = new Date(weekStart)
    current.setDate(current.getDate() + (offset * 7))
    setWeekStart(current.toISOString().split('T')[0])
  }
}

```

```

}

return (
  <div className="space-y-4">
    {/* Week Navigation */}
    <div className="flex items-center justify-between bg-stone-800 text-white rounded-lg
p-4">
      <button onClick={() => navigateWeek(-1)}
        className="px-4 py-2 bg-amber-600 text-white rounded hover:bg-amber-700">
        ← Previous Week
      </button>
      <div className="text-center">
        <h2 className="text-xl font-semibold">
          Week of {formatDate(new Date(weekStart))}
        </h2>
      </div>
      <button onClick={() => navigateWeek(1)}
        className="px-4 py-2 bg-amber-600 text-white rounded hover:bg-amber-700">
        Next Week →
      </button>
    </div>

    {/* Schedule Table */}
    <div className="overflow-x-auto">
      <table className="w-full border-collapse">
        <thead>
          <tr>
            <th className="p-3 bg-stone-800 text-white">Shift</th>
            {DAYS_OF_WEEK.map((day, i) => (
              <th key={day} className="p-3 bg-stone-800 text-white text-center">
                <div>{day}</div>
                <div className="text-xs text-stone-300">
                  {formatDate(getDateFromDayOfWeek(weekStart, i))}
                </div>
              </th>
            ))}
          </tr>
        </thead>
        <tbody>
          {[ 'opening', 'mid', 'closing' ].map((shiftType) => (
            <tr key={shiftType} className="border-b border-stone-200">
              <td className="p-3 font-semibold bg-amber-50 text-stone-700 whitespace-
nowrap">
                {shiftType === 'opening' ? 'Opening (10am–4pm)'
                  : shiftType === 'mid' ? 'Mid (12pm–6pm)'
                  : 'Closing (2pm–8/9pm)'}
              </td>
              {DAYS_OF_WEEK.map((_, dayIndex) => (
                <td key={dayIndex} className="p-2 text-center border border-stone-100">
                  <ShiftCell
                    dayOfWeek={dayIndex}
                    shiftType={shiftType}
                    currentEmployee={getEmployeeForShift(dayIndex, shiftType)}
                    employees={employees}
                    onUpdate={handleShiftUpdate}
                    isSaturday={dayIndex === 6}
                  />
                </td>
              ))}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>

```

```
)  
}
```

Step 31: Wire up the main page.

Replace `app/page.tsx`:

```
// app/page.tsx  
import ScheduleGrid from '@components/ScheduleGrid'  
  
export default function Home() {  
  return (  
    <main className="min-h-screen bg-stone-50 p-8">  
      <div className="max-w-7xl mx-auto">  
        <header className="mb-8">  
          <h1 className="text-3xl font-bold text-stone-800">  
            Fabulosa Books  
          </h1>  
          <p className="text-stone-500 mt-1">Employee Schedule</p>  
        </header>  
        <ScheduleGrid />  
      </div>  
    </main>  
  )  
}
```

Step 32: Run and verify.

```
npm run dev
```

Open `http://localhost:3000`. You should see the schedule grid with week navigation and 21 “Unassigned” cells. Click any cell and pick an employee to verify the assignment works.

[IMAGE: Screenshot of the working schedule grid at localhost:3000 — the header says “Fabulosa Books”, the grid shows days across the top and shifts down the side, with dropdown cells. One or two cells have been assigned to show the UI in action.]

The Turning Point

The schedule app works. You can click each cell, pick an employee, and build a week’s schedule one shift at a time. Push it to GitHub and Vercel deploys it automatically.

But right now, filling it is manual. Twenty-one cells, twenty-one clicks, twenty-one decisions. Alvin still has to do all the thinking: Who’s available? Is this fair? Is the store covered?

Let’s teach the app to think.

Part 5: Your First Agent — The Availability Agent

From Clicking to Asking

Right now, filling the schedule means clicking twenty-one dropdowns. What if you could just *ask* the app a question instead?

“Who’s available Thursday?” “Is anyone free to cover the closing shift on Saturday?” “Which on-call staff haven’t worked this week?”

That’s what an agent does. It takes a natural-language question, uses its tools to find the answer, and responds intelligently. We’re going to build the Availability Agent—the first member of our specialist team.

Building Your First Tool

Step 33: Create the tools directory and the schedule query tool.

Create `lib/tools/schedule-tools.ts`:

```
// lib/tools/schedule-tools.ts

import { tool } from 'ai'
import { z } from 'zod'
import { getAllEmployees, getWeekSchedule } from '@lib/db'
import { getWeekStartDate } from '@lib/utils'

export const getEmployeesTool = tool({
  description: 'Get all employees at Fabulosa Books with their role (regular or on-call)',
  inputSchema: z.object({}),
  execute: async () => {
    const employees = await getAllEmployees()
    return employees.map(emp => ({
      name: emp.name,
      type: emp.is_on_call ? 'on-call' : 'regular',
    }))
  },
})

export const getScheduleTool = tool({
  description: 'Get the current schedule for a specific week. Returns all shift assignments for that week.',
  inputSchema: z.object({
    weekStart: z.string()
      .describe('The start date of the week (Sunday) in YYYY-MM-DD format. Use the current week if not specified.'),
  }),
  execute: async ({ weekStart }) => {
    const schedule = await getWeekSchedule(weekStart)
    return {
      weekStart,
      assignments: schedule.map(entry => ({
        day: entry.day_of_week,
        shift: entry.shift_type,
        employee: entry.employee_name,
      })),
      totalAssigned: schedule.filter(e => e.employee_name).length,
      totalSlots: 21,
    }
  },
})

export const checkAvailabilityTool = tool({
  description: 'Check which employees are available (not already scheduled) for a specific day in a given week.',
  inputSchema: z.object({
    weekStart: z.string().describe('Week start date (Sunday) in YYYY-MM-DD format'),
    dayOfWeek: z.number().min(0).max(6).describe('Day of week: 0=Sunday, 1=Monday, ..., 6=Saturday'),
  })
```

```

    }},
    execute: async ({ weekStart, dayOfWeek }) => {
      const [employees, schedule] = await Promise.all([
        getAllEmployees(),
        getWeekSchedule(weekStart),
      ])

      const assignedOnDay = schedule
        .filter(s => s.day_of_week === dayOfWeek && s.employee_name)
        .map(s => s.employee_name)

      const available = employees.filter(emp => !assignedOnDay.includes(emp.name))

      return {
        dayOfWeek,
        weekStart,
        available: available.map(emp => ({
          name: emp.name,
          type: emp.is_on_call ? 'on-call' : 'regular',
        })),
        alreadyScheduled: assignedOnDay,
      }
    },
  })
})

```

Let's break down what's happening here:

- **getEmployeesTool** — returns the full roster with regular/on-call designation
- **getScheduleTool** — returns a week's assignments with a summary of how many slots are filled
- **checkAvailabilityTool** — cross-references the roster against the schedule to find who's free on a given day

Each tool has a description (so the AI knows when to use it), an input schema (so the AI knows what to provide), and an execute function (your actual code).

[IMAGE: A diagram showing the three tools as boxes, with inputs on the left and outputs on the right. getEmployeesTool: {} → [{name, type}]. getScheduleTool: {weekStart} → {assignments, counts}. checkAvailabilityTool: {weekStart, dayOfWeek} → {available, alreadyScheduled}.

Caption: "Three tools, three specific capabilities."

Building the Agent

Step 34: Create the Availability Agent.

Create **lib/agents/availability-agent.ts**:

```

// lib/agents/availability-agent.ts

import { ToolLoopAgent, InferAgentUIMessage } from 'ai'
import { getEmployeesTool, getScheduleTool, checkAvailabilityTool } from
'../tools/schedule-tools'

export const availabilityAgent = new ToolLoopAgent({
  model: 'anthropic/claude-sonnet-4.5',
  instructions: `You are the Availability Agent for Fabulosa Books, an LGBTQIA+ bookstore
in San Francisco's Castro neighborhood.

Your job is to answer questions about employee availability and the current schedule.

```

```

The store has 8 employees:
- 6 regular staff: Alvin (owner), Melissa, Marcus, Dylan, Joel, Billy
- 2 on-call staff: Johnny Ray, Carly

The store operates 10am-8pm daily (10am-9pm Saturdays) with three shifts:
- Opening: 10am-4pm
- Mid: 12pm-6pm
- Closing: 2pm-8pm (2pm-9pm Saturdays)

Each shift is assigned to one employee. Use your tools to check the actual schedule data
before answering. Never guess or make up availability – always query the database.

When reporting availability, distinguish between regular and on-call staff. On-call staff
should generally only be suggested when regular staff aren't available.`
tools: {
  getEmployees: getEmployeesTool,
  getSchedule: getScheduleTool,
  checkAvailability: checkAvailabilityTool,
},
})

// Export the type for use in the UI
export type AvailabilityAgentUIMessage = InferAgentUIMessage<typeof availabilityAgent>

```

Let's unpack the key parts:

- **ToolLoopAgent** — This is the AI SDK's agent primitive. It wraps a model with instructions and tools, and manages the “think → use tool → observe → think again” loop automatically.
- **model: 'anthropic/claude-sonnet-4'** — This uses the Vercel AI Gateway. The string `anthropic/claude-sonnet-4` routes through the gateway to Anthropic's Claude. You could swap to `openai/gpt-4o` or `google/gemini-2.5-pro` without changing anything else.
- **instructions** — The agent's identity and rules. It knows about Fabulosa, the team, the shifts. These instructions shape how the agent reasons.
- **tools** — The capabilities the agent can use. It can get employees, check the schedule, and check availability.
- **InferAgentUIMessage** — This extracts the TypeScript type for UI messages, giving you type-safe tool results in your React components.

[IMAGE: An annotated code snippet of the ToolLoopAgent definition, with callout boxes explaining each part: model (which AI to use), instructions (the agent's personality and rules), tools (what it can do), and the exported type (for the UI). Caption: “Anatomy of an AI SDK agent.”]

Wiring the Agent to an API Route

Step 35: Create the chat API route.

Create `app/api/chat/route.ts`:

```

// app/api/chat/route.ts

import { createAgentUIStreamResponse } from 'ai'
import { availabilityAgent } from '@lib/agents/availability-agent'

```

```
export async function POST(request: Request) {
  const { messages } = await request.json()

  return createAgentUIStreamResponse({
    agent: availabilityAgent,
    uiMessages: messages,
  })
}
```

That's it. Three lines of meaningful code. `createAgentUIStreamResponse` takes the agent and the conversation history, runs the agent's tool loop, and streams the response back to the browser in a format that `useChat` understands.

[IMAGE: A flow diagram showing: Browser (useChat) → POST /api/chat → createAgentUIStreamResponse → ToolLoopAgent runs tool loop → streams response back → Browser updates in real-time. Caption: "The agent runs server-side. The response streams to the browser."]

Adding Chat to the UI

Step 36: Create the ChatPanel component.

Create `components/ChatPanel.tsx`:

```
// components/ChatPanel.tsx
'use client'

import { useChat } from '@ai-sdk/react'
import { DefaultChatTransport } from 'ai'
import { useState } from 'react'
import type { AvailabilityAgentUIMessage } from '@lib/agents/availability-agent'

const transport = new DefaultChatTransport({ api: '/api/chat' })

export default function ChatPanel() {
  const [input, setInput] = useState('')
  const { messages, sendMessage, isLoading } = useChat<AvailabilityAgentUIMessage>({
    transport,
  })

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault()
    if (!input.trim()) return
    sendMessage({ text: input })
    setInput('')
  }

  return (
    <div className="bg-white rounded-lg border border-stone-200 shadow-sm">
      <div className="p-4 bg-stone-800 text-white rounded-t-lg">
        <h3 className="font-semibold">Schedule Assistant</h3>
        <p className="text-sm text-stone-300">Ask about availability and shifts</p>
      </div>

      { /* Messages */ }
      <div className="h-96 overflow-y-auto p-4 space-y-4">
        {messages.length === 0 && (
          <div className="text-stone-400 text-sm text-center mt-8">
            Try asking: "Who's available this Thursday?"
          </div>
        )}
      </div>
    </div>
  )
}
```

```

    ))
    {messages.map(message => (
      <div
        key={message.id}
        className={flex ${message.role === 'user' ? 'justify-end' : 'justify-start'}}
      >
        <div className={max-w-[80%] rounded-lg p-3 text-sm ${
          message.role === 'user'
            ? 'bg-amber-500 text-white'
            : 'bg-stone-100 text-stone-800'
          }
        >
          {message.parts.map((part, i) => {
            switch (part.type) {
              case 'text':
                return <span key={i}>{part.text}</span>
              case 'tool-getEmployees':
              case 'tool-getSchedule':
              case 'tool-checkAvailability':
                return (
                  <div key={i} className="text-xs text-stone-400 italic my-1">
                    {part.state === 'output-available'
                      ? `✓ Checked ${part.type.replace('tool-', '')}`
                      : `⌚ Checking ${part.type.replace('tool-', '')}...`}
                  </div>
                )
              default:
                return null
            }
          })}
        </div>
      </div>
    ))}
  </div>

  {/* Input */}
  <form onSubmit={handleSubmit} className="p-4 border-t border-stone-200">
    <div className="flex gap-2">
      <input
        type="text"
        value={input}
        onChange={e => setInput(e.target.value)}
        placeholder="Ask about the schedule..."
        className="flex-1 px-3 py-2 border border-stone-300 rounded-lg text-sm
          focus:outline-none focus:ring-2 focus:ring-amber-500"
        disabled={isLoading}
      />
      <button
        type="submit"
        disabled={isLoading || !input.trim()}
        className="px-4 py-2 bg-amber-600 text-white rounded-lg text-sm
          hover:bg-amber-700 disabled:opacity-50"
      >
        Ask
      </button>
    </div>
  </form>
</div>
)
}

```

Notice how the tool parts render: when the agent calls a tool, the UI shows “⌚ Checking...” while it runs, then “✓ Checked” when it completes. The user can see the agent *working*—not just waiting for a final answer.

[IMAGE: Screenshot of the ChatPanel in action — the user has asked “Who’s available this Thursday?” and the agent’s response shows “✓ Checked getEmployees”, “✓ Checked getSchedule”, followed by a text response listing available employees. The chat panel has the warm amber/stone color scheme.]

Step 37: Add the ChatPanel to the main page.

Update `app/page.tsx` to include the chat panel alongside the schedule grid:

```
// app/page.tsx
import ScheduleGrid from '@components/ScheduleGrid'
import ChatPanel from '@components/ChatPanel'

export default function Home() {
  return (
    <main className="min-h-screen bg-stone-50 p-8">
      <div className="max-w-7xl mx-auto">
        <header className="mb-8">
          <h1 className="text-3xl font-bold text-stone-800">
            Fabulosa Books
          </h1>
          <p className="text-stone-500 mt-1">Employee Schedule</p>
        </header>
        <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
          <div className="lg:col-span-2">
            <ScheduleGrid />
          </div>
          <div>
            <ChatPanel />
          </div>
        </div>
      </div>
    </main>
  )
}
```

Step 38: Test your first agent.

```
npm run dev
```

Open `http://localhost:3000`. You should see the schedule grid on the left and the chat panel on the right. Try these prompts:

- “Who’s available this Thursday?”
- “Show me this week’s schedule”
- “Which on-call staff are available Saturday?”

[IMAGE: Full-page screenshot showing the schedule grid (left, ~2/3 width) and the chat panel (right, ~1/3 width) side by side. The chat shows a multi-turn conversation about availability. Caption: “The schedule grid and the AI assistant, side by side.”]

The agent queries the database, reasons about the results, and responds in natural language. It’s not guessing—it’s checking real data with real tools.

Step 39: Push and deploy.

```
git add -A
git commit -m "Add Availability Agent with chat interface"
git push
```

Vercel deploys automatically. In about a minute, your live app has an AI scheduling assistant.

[IMAGE: Screenshot of the Vercel deployment log showing the build succeeding, with the deployment URL highlighted]

What Just Happened

You built an AI agent. Not a chatbot that makes things up—an agent that:

1. Receives a natural-language question
2. Decides which tools to call
3. Queries a real database
4. Reasons about the results
5. Responds with an accurate, helpful answer

Billy shows Alvin. “That’s cool,” Alvin says. “But it only tells me who’s available. It doesn’t know if the schedule is *fair*.”

He’s right. The Availability Agent answers “who *can* work.” But it doesn’t answer “who *should* work.” For that, we need more agents.

Part 6: The Specialist Team — Multi-Agent Orchestration

One Agent Isn’t Enough

The Availability Agent is useful, but it has blind spots. It can tell you Dylan is available Thursday, but it doesn’t know that Dylan has already worked five shifts this week and giving him a sixth would be unfair. It doesn’t know that Marcus has had closing shifts three weeks running. It doesn’t check whether the store actually has enough people on Saturday.

Each of these is a different kind of reasoning. And each one deserves its own specialist.

The Fairness Agent

Step 40: Create the fairness tools.

Create `lib/tools/fairness-tools.ts`:

```
// lib/tools/fairness-tools.ts

import { tool } from 'ai'
import { z } from 'zod'
import { getWeekSchedule, getAllEmployees } from '@lib/db'
import { getWeekStartDate } from '@lib/utils'

export const getShiftDistributionTool = tool({
  description: 'Analyze how shifts are distributed across employees for a given week. Returns the number of shifts each employee has been assigned.',
  inputSchema: z.object({
```

```

    weekStart: z.string().describe('Week start date (Sunday) in YYYY-MM-DD format'),
  }},
  execute: async ({ weekStart }) => {
    const [employees, schedule] = await Promise.all([
      getAllEmployees(),
      getWeekSchedule(weekStart),
    ])

    const distribution: Record<string, { total: number; opening: number; mid: number;
closing: number }> = {}

    employees.forEach(emp => {
      distribution[emp.name] = { total: 0, opening: 0, mid: 0, closing: 0 }
    })

    schedule.forEach(entry => {
      if (entry.employee_name && distribution[entry.employee_name]) {
        distribution[entry.employee_name].total++
        distribution[entry.employee_name][entry.shift_type as 'opening' | 'mid' |
'closing']++
      }
    })

    return { weekStart, distribution }
  },
})

export const getMultiWeekHistoryTool = tool({
  description: 'Get shift history across multiple weeks for fairness analysis. Shows how
many shifts each employee has worked over the specified number of past weeks.',
  inputSchema: z.object({
    weeksBack: z.number().min(1).max(8)
      .describe('Number of past weeks to analyze (1-8)'),
  }),
  execute: async ({ weeksBack }) => {
    const history: Record<string, { totalShifts: number; closingShifts: number;
saturdayShifts: number }> = {}
    const employees = await getAllEmployees()

    employees.forEach(emp => {
      history[emp.name] = { totalShifts: 0, closingShifts: 0, saturdayShifts: 0 }
    })

    for (let i = 0; i < weeksBack; i++) {
      const d = new Date()
      d.setDate(d.getDate() - (i * 7))
      const weekStart = getWeekStartDate(d)
      const schedule = await getWeekSchedule(weekStart)

      schedule.forEach(entry => {
        if (entry.employee_name && history[entry.employee_name]) {
          history[entry.employee_name].totalShifts++
          if (entry.shift_type === 'closing') {
            history[entry.employee_name].closingShifts++
          }
          if (entry.day_of_week === 6) {
            history[entry.employee_name].saturdayShifts++
          }
        }
      })
    }

    return {
      weeksAnalyzed: weeksBack,
      history,
    }
  }
})

```

```

    fairnessNotes: analyzeFairness(history),
  },
},
})

function analyzeFairness(
  history: Record<string, { totalShifts: number; closingShifts: number; saturdayShifts:
number }>
) {
  const notes: string[] = []
  const entries = Object.entries(history)

  const totals = entries.map(([, h]) => h.totalShifts).filter(t => t > 0)
  if (totals.length > 0) {
    const avg = totals.reduce((a, b) => a + b, 0) / totals.length
    const max = Math.max(...totals)
    const min = Math.min(...totals)
    if (max - min > 3) {
      const overworked = entries.filter(([, h]) => h.totalShifts === max).map(([name]) =>
name)
      const underworked = entries.filter(([, h]) => h.totalShifts === min).map(([name]) =>
name)
      notes.push(`Imbalance detected: ${overworked.join(', ')} have ${max} shifts while $
{underworked.join(', ')} have ${min}`)
    }
  }

  entries.forEach(([name, h]) => {
    if (h.closingShifts >= 3) {
      notes.push(`${name} has had ${h.closingShifts} closing shifts – consider rotating`)
    }
  })

  return notes
}

```

[IMAGE: An example output of the fairness analysis — a bar chart or table showing shift distribution across employees, with Marcus highlighted as having 3 closing shifts and Dylan having 0. Caption: “The Fairness Agent sees imbalances humans might miss.”]

Step 41: Create the Fairness Agent.

Create `lib/agents/fairness-agent.ts`:

```

// lib/agents/fairness-agent.ts

import { ToolLoopAgent, InferAgentUIMessage } from 'ai'
import { getShiftDistributionTool, getMultiWeekHistoryTool } from '../tools/fairness-tools'

export const fairnessAgent = new ToolLoopAgent({
  model: 'anthropic/claude-sonnet-4.5',
  instructions: `You are the Fairness Agent for Fabulosa Books.

Your job is to analyze whether shift assignments are equitable across employees. Fabulosa
Books cares deeply about treating everyone fairly – this is a core value of the store.

Fairness principles:
- Total shifts should be roughly equal across regular staff (±1 per week is acceptable)
- No one should get stuck with closing shifts repeatedly – rotate them
- Saturday shifts (which run an extra hour) should be shared
- On-call staff (Johnny Ray, Carly) should only be used when regular staff can't cover
- Alvin (the owner) and Melissa (senior staff) should not be disproportionately burdened

```

When asked to evaluate fairness, always check the multi-week history – a single week snapshot can be misleading. Look for patterns over 3-4 weeks.

Be specific in your recommendations: don't just say "distribute more evenly" – say "Give Dylan a closing shift this week since Marcus has had closing three weeks in a row."

```
tools: {
  getShiftDistribution: getShiftDistributionTool,
  getMultiWeekHistory: getMultiWeekHistoryTool,
},
})
```

```
export type FairnessAgentUIMessage = InferAgentUIMessage<typeof fairnessAgent>
```

Notice how the instructions encode Fabulosa's *values*, not just rules. The agent knows that fairness matters to the store's identity. It's not just checking numbers—it's reasoning about equity.

The Coverage Agent

Step 42: Create the coverage tools.

Create `lib/tools/coverage-tools.ts`:

```
// lib/tools/coverage-tools.ts

import { tool } from 'ai'
import { z } from 'zod'
import { getWeekSchedule } from '@lib/db'

export const validateCoverageTool = tool({
  description: 'Validate whether a week\'s schedule meets all staffing requirements. Checks that every shift has an assigned employee and that Saturday closing is correctly set.',
  inputSchema: z.object({
    weekStart: z.string().describe('Week start date (Sunday) in YYYY-MM-DD format'),
  }),
  execute: async ({ weekStart }) => {
    const schedule = await getWeekSchedule(weekStart)

    const gaps: { day: number; shift: string }[] = []
    const issues: string[] = []

    for (let day = 0; day < 7; day++) {
      for (const shift of ['opening', 'mid', 'closing']) {
        const entry = schedule.find(
          s => s.day_of_week === day && s.shift_type === shift
        )
        if (!entry || !entry.employee_name) {
          gaps.push({ day, shift })
        }
      }
    }

    // Check for double-booking (same person, same day, multiple shifts)
    for (let day = 0; day < 7; day++) {
      const dayEntries = schedule.filter(s => s.day_of_week === day && s.employee_name)
      const names = dayEntries.map(s => s.employee_name)
      const duplicates = names.filter((name, i) => names.indexOf(name) !== i)
      if (duplicates.length > 0) {
        issues.push(`Double-booked on day ${day}: ${[...new Set(duplicates)].join(', ')}`)
      }
    }

    return {

```

```

weekStart,
isFullyCovered: gaps.length === 0,
gaps,
issues,
totalAssigned: schedule.filter(s => s.employee_name).length,
totalSlots: 21,
},
},
})

```

Step 43: Create the Coverage Agent.

Create `lib/agents/coverage-agent.ts`:

```

// lib/agents/coverage-agent.ts

import { ToolLoopAgent, InferAgentUIMessage } from 'ai'
import { validateCoverageTool } from '../tools/coverage-tools'

export const coverageAgent = new ToolLoopAgent({
  model: 'anthropic/claude-sonnet-4.5',
  instructions: `You are the Coverage Agent for Fabulosa Books.

Your job is to validate that the store's schedule meets all staffing requirements.

Coverage rules:
- Every day must have exactly 3 shifts: opening (10am-4pm), mid (12pm-6pm), closing (2pm-8pm, or 2pm-9pm on Saturday)
- Each shift must have exactly one employee assigned
- No employee should be double-booked (two shifts on the same day)
- A fully covered week = 21 assigned shifts (3 shifts × 7 days)

When reporting gaps, be specific about which day and shift is uncovered. When reporting issues, explain the impact (e.g., "No one is assigned to close on Saturday – the store would need to close an hour early or find emergency coverage").`,
  tools: {
    validateCoverage: validateCoverageTool,
  },
})

export type CoverageAgentUIMessage = InferAgentUIMessage<typeof coverageAgent>

```

[IMAGE: A visual showing all three agents side by side with their specialties: Availability (database icon + “Who’s free?”), Fairness (balance scale icon + “Is it equitable?”), Coverage (checklist icon + “Is the store staffed?”). Caption: “Three specialists, three perspectives on the same schedule.”]

The Coordinator: Orchestrating It All

This is the heart of multi-agent orchestration. The Schedule Coordinator is an agent whose tools are the other agents.

Step 44: Create the coordinator agent.

Create `lib/agents/coordinator-agent.ts`:

```

// lib/agents/coordinator-agent.ts

import { ToolLoopAgent, InferAgentUIMessage, tool } from 'ai'
import { z } from 'zod'
import { availabilityAgent } from '../availability-agent'

```

```

import { fairnessAgent } from './fairness-agent'
import { coverageAgent } from './coverage-agent'
import { updateSchedule } from '@/lib/db'
import { getWeekStartDate } from '@/lib/utils'

// Wrap each specialist as a tool the coordinator can call
const checkAvailabilitySubagent = tool({
  description: 'Ask the Availability Agent about employee availability and current schedule state. Use this when you need to know who is free on a specific day or what the current schedule looks like.',
  inputSchema: z.object({
    question: z.string().describe('The availability question to answer'),
  }),
  execute: async ({ question }) => {
    const result = await availabilityAgent.generate({
      messages: [{ role: 'user', content: question }],
    })
    return { answer: result.text }
  },
})

const checkFairnessSubagent = tool({
  description: 'Ask the Fairness Agent to evaluate shift distribution equity. Use this when you need to ensure a schedule or assignment is fair to all employees.',
  inputSchema: z.object({
    question: z.string().describe('The fairness question to evaluate'),
  }),
  execute: async ({ question }) => {
    const result = await fairnessAgent.generate({
      messages: [{ role: 'user', content: question }],
    })
    return { answer: result.text }
  },
})

const validateCoverageSubagent = tool({
  description: 'Ask the Coverage Agent to validate whether the store is properly staffed. Use this to check for gaps, double-bookings, or incomplete schedules.',
  inputSchema: z.object({
    question: z.string().describe('The coverage question to validate'),
  }),
  execute: async ({ question }) => {
    const result = await coverageAgent.generate({
      messages: [{ role: 'user', content: question }],
    })
    return { answer: result.text }
  },
})

const assignShiftTool = tool({
  description: 'Assign an employee to a specific shift. Use this after determining the best assignment through availability, fairness, and coverage checks.',
  inputSchema: z.object({
    weekStart: z.string().describe('Week start date (Sunday) in YYYY-MM-DD format'),
    dayOfWeek: z.number().min(0).max(6).describe('Day of week: 0=Sunday through 6=Saturday'),
    shiftType: z.enum(['opening', 'mid', 'closing']).describe('The shift type'),
    employeeName: z.string().describe('The employee name to assign'),
  }),
  execute: async ({ weekStart, dayOfWeek, shiftType, employeeName }) => {
    await updateSchedule(weekStart, dayOfWeek, shiftType, employeeName)
    return { success: true, assigned: `${employeeName} → ${shiftType} on day ${dayOfWeek}` }
  },
})

```

```

export const coordinatorAgent = new ToolLoopAgent({
  model: 'anthropic/claude-sonnet-4.5',
  instructions: `You are the Schedule Coordinator for Fabulosa Books, an LGBTQIA+ bookstore
in San Francisco's Castro neighborhood.

You coordinate scheduling by delegating to three specialist agents:
1. **Availability Agent** – checks who is free and when
2. **Fairness Agent** – ensures equitable shift distribution
3. **Coverage Agent** – validates store staffing requirements

When handling scheduling requests:
1. First, check availability to understand the current state
2. Then, consult the Fairness Agent to understand distribution patterns
3. Make assignment decisions that balance availability, fairness, and coverage
4. After making assignments, validate coverage to ensure no gaps
5. Report the final result clearly

You can also assign shifts directly using the assignShift tool.

Important guidelines:
- Always consult at least the Availability Agent before making suggestions
- For full schedule creation, consult all three specialists
- Prefer regular staff over on-call staff
- Be transparent about trade-offs: if perfect fairness conflicts with coverage, explain the
compromise
- Present proposed schedules clearly in a table format when possible

Today's date can be used to calculate the current week start date: ${getWeekStartDate()}`,
  tools: {
    checkAvailability: checkAvailabilitySubagent,
    checkFairness: checkFairnessSubagent,
    validateCoverage: validateCoverageSubagent,
    assignShift: assignShiftTool,
  },
})

export type CoordinatorAgentUIMessage = InferAgentUIMessage<typeof coordinatorAgent>

```

This is the key pattern. The coordinator's tools are the other agents. When it calls `checkAvailability`, it's actually running the Availability Agent as a subagent — a full agent with its own tools and reasoning loop. The coordinator receives a text summary back, reasons about it, and decides what to do next.

[IMAGE: A detailed orchestration diagram showing: User message flows into the Coordinator Agent → Coordinator decides to call `checkAvailability` → Availability Agent runs its own tool loop (`getEmployees`, `getSchedule`) → returns summary to Coordinator → Coordinator calls `checkFairness` → Fairness Agent runs its tool loop → returns → Coordinator calls `assignShift` → database updated → Coordinator responds to user. Each agent is shown as a distinct box with its own internal tool loop. Caption: "Agents calling agents — this is multi-agent orchestration."]

Step 45: Update the chat API route to use the coordinator.

Update `app/api/chat/route.ts`:

```

// app/api/chat/route.ts

import { createAgentUIStreamResponse } from 'ai'
import { coordinatorAgent } from '@lib/agents/coordinator-agent'

export async function POST(request: Request) {

```

```
const { messages } = await request.json()

return createAgentUIStreamResponse({
  agent: coordinatorAgent,
  uiMessages: messages,
})
}
```

One line changed: `availabilityAgent` → `coordinatorAgent`. The chat interface is the same, but now the coordinator delegates to all three specialists.

Updating the Chat UI

Step 46: Update ChatPanel to show agent activity.

Update `components/ChatPanel.tsx` to handle the coordinator's tool calls:

```
// components/ChatPanel.tsx
'use client'

import { useChat } from '@ai-sdk/react'
import { DefaultChatTransport, isToolUIPart } from 'ai'
import { useState } from 'react'
import type { CoordinatorAgentUIMessage } from '@lib/agents/coordinator-agent'

const transport = new DefaultChatTransport({ api: '/api/chat' })

const AGENT_LABELS: Record<string, string> = {
  'tool-checkAvailability': '📅 Availability Agent',
  'tool-checkFairness': '⚖️ Fairness Agent',
  'tool-validateCoverage': '✅ Coverage Agent',
  'tool-assignShift': '👤 Assigning Shift',
}

export default function ChatPanel({ onScheduleChange }: { onScheduleChange?: () => void }) {
  const [input, setInput] = useState('')
  const { messages, sendMessage, isLoading } = useChat<CoordinatorAgentUIMessage>({
    transport,
    onFinish: () => onScheduleChange?.(),
  })

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault()
    if (!input.trim()) return
    sendMessage({ text: input })
    setInput('')
  }

  return (
    <div className="bg-white rounded-lg border border-stone-200 shadow-sm">
      <div className="p-4 bg-stone-800 text-white rounded-t-lg">
        <h3 className="font-semibold">Schedule Coordinator</h3>
        <p className="text-sm text-stone-300">Multi-agent scheduling assistant</p>
      </div>

      { /* Messages */ }
      <div className="h-[500px] overflow-y-auto p-4 space-y-4">
        {messages.length === 0 && (
          <div className="text-stone-400 text-sm text-center mt-8 space-y-2">
            <p>Try asking:</p>
            <p className="italic">"Schedule next week"</p>
            <p className="italic">"Is this week's schedule fair?"</p>
          </div>
        )}
      </div>
    </div>
  )
}
```

```

        <p className="italic">"Find someone to cover Tuesday closing"</p>
      </div>
    )}
    {messages.map(message => (
      <div
        key={message.id}
        className={`flex ${message.role === 'user' ? 'justify-end' : 'justify-start'}}`
      >
        <div className={`max-w-[85%] rounded-lg p-3 text-sm ${
          message.role === 'user'
            ? 'bg-amber-500 text-white'
            : 'bg-stone-100 text-stone-800'
          }`} >
          {message.parts.map((part, i) => {
            if (part.type === 'text') {
              return <span key={i} className="whitespace-pre-wrap">{part.text}</span>
            }
            if (isToolUIPart(part)) {
              const label = AGENT_LABELS[part.type] || part.type
              return (
                <div key={i} className="text-xs text-stone-400 italic my-1 flex items-
center gap-1">
                  {part.state === 'output-available' ? '✓' : '⌚'} {label}
                </div>
              )
            }
            return null
          })}
        </div>
      </div>
    )})
    {isLoading && (
      <div className="text-stone-400 text-sm animate-pulse">
        Agents are working...
      </div>
    )}
  </div>

  {/* Input */}
  <form onSubmit={handleSubmit} className="p-4 border-t border-stone-200">
    <div className="flex gap-2">
      <input
        type="text"
        value={input}
        onChange={e => setInput(e.target.value)}
        placeholder="Ask the coordinator..."
        className="flex-1 px-3 py-2 border border-stone-300 rounded-lg text-sm
          focus:outline-none focus:ring-2 focus:ring-amber-500"
        disabled={isLoading}
      />
      <button
        type="submit"
        disabled={isLoading || !input.trim()}
        className="px-4 py-2 bg-amber-600 text-white rounded-lg text-sm
          hover:bg-amber-700 disabled:opacity-50"
      >
        Ask
      </button>
    </div>
  </form>
</div>
)
}

```

The key upgrade: the chat panel now shows *which agent* is working. When the coordinator delegates to the Fairness Agent, you see “⌚ ⚖️ Fairness Agent” in the response. When it completes, it becomes “✓ ⚖️ Fairness Agent.” The user can watch the orchestration happening in real-time.

[IMAGE: Screenshot of the chat panel showing a multi-agent interaction. The user asked “Is this week’s schedule fair?” The response shows: “✓ 📅 Availability Agent” → “✓ ⚖️ Fairness Agent” → a text response explaining that Marcus has had 3 closing shifts in a row and recommending rotation. Caption: “Watching agents collaborate in real-time.”]

Step 47: Connect schedule changes back to the grid.

Update `app/page.tsx` so the schedule grid refreshes when the coordinator assigns shifts:

```
// app/page.tsx
'use client'

import { useState, useCallback } from 'react'
import ScheduleGrid from '@components/ScheduleGrid'
import ChatPanel from '@components/ChatPanel'

export default function Home() {
  const [refreshKey, setRefreshKey] = useState(0)

  const handleScheduleChange = useCallback(() => {
    setRefreshKey(k => k + 1)
  }, [])

  return (
    <main className="min-h-screen bg-stone-50 p-8">
      <div className="max-w-7xl mx-auto">
        <header className="mb-8">
          <h1 className="text-3xl font-bold text-stone-800">
            Fabulosa Books
          </h1>
          <p className="text-stone-500 mt-1">Employee Schedule – Multi-Agent
Coordinator</p>
        </header>
        <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
          <div className="lg:col-span-2">
            <ScheduleGrid key={refreshKey} />
          </div>
          <div>
            <ChatPanel onScheduleChange={handleScheduleChange} />
          </div>
        </div>
      </div>
    </main>
  )
}
```

Step 48: Test multi-agent orchestration.

```
npm run dev
```

Try these prompts:

- **“Is this week’s schedule fair?”** — The coordinator calls the Availability Agent (to get the current schedule), then the Fairness Agent (to analyze distribution). You’ll see both agents activate in the chat panel.

- **“Find someone to cover Tuesday closing”** — The coordinator checks availability for Tuesday, checks fairness to pick the best candidate, and suggests an assignment.
- **“Schedule next week”** — The full orchestration: availability → fairness → coverage → assignments. The coordinator delegates to all three specialists and synthesizes a complete schedule.

[IMAGE: Full-page screenshot showing the complete app — schedule grid on the left with some shifts filled in by the coordinator, chat panel on the right showing a multi-step orchestration with all three agent indicators (Availability ✓, Fairness ✓, Coverage ✓) and a detailed schedule recommendation. Caption: “The full multi-agent system in action.”]

Step 49: Push and deploy.

```
git add -A
git commit -m "Add multi-agent orchestration: Fairness, Coverage, and Coordinator agents"
git push
```

[IMAGE: Screenshot of the deployed app on Vercel, with the URL visible in the browser bar]

What You’ve Built

Take a moment to appreciate what just happened. You’ve built a multi-agent system where:

1. A **Coordinator Agent** receives natural-language scheduling requests
2. It delegates to an **Availability Agent** that queries the database
3. It consults a **Fairness Agent** that analyzes historical patterns
4. It validates with a **Coverage Agent** that checks staffing rules
5. It synthesizes the results and either makes recommendations or assigns shifts directly
6. The UI shows the orchestration happening in real-time
7. The schedule grid updates automatically when shifts are assigned

That’s multi-agent orchestration. Not one monolithic AI trying to do everything, but a team of specialists coordinating through a leader—just like Alvin delegates to his team at the bookstore.

[IMAGE: A final architecture diagram showing the complete system: Browser (Schedule Grid + Chat Panel) → Next.js API Route → Coordinator Agent → three specialist agents (each with their own tools) → Neon Database. All deployed on Vercel. Caption: “The complete multi-agent architecture.”]

Part 7: Making It Real

The Communication Agent

There’s one more agent worth building: the Communication Agent. While the other three handle the *thinking*, this one handles the *talking*—drafting human-readable summaries and notifications.

Step 50: Create the Communication Agent.

Create `lib/tools/communication-tools.ts`:

```
// lib/tools/communication-tools.ts

import { tool } from 'ai'
import { z } from 'zod'
import { getWeekSchedule, getAllEmployees } from '@lib/db'
import { DAYS_OF_WEEK } from '@lib/utils'

export const formatWeekSummaryTool = tool({
  description: 'Generate a formatted summary of a week\'s schedule, suitable for posting or sending to the team.',
  inputSchema: z.object({
    weekStart: z.string().describe('Week start date (Sunday) in YYYY-MM-DD format'),
  }),
  execute: async ({ weekStart }) => {
    const [schedule, employees] = await Promise.all([
      getWeekSchedule(weekStart),
      getAllEmployees(),
    ])

    const byDay: Record<number, Record<string, string>> = {}
    for (let d = 0; d < 7; d++) {
      byDay[d] = {}
      for (const shift of ['opening', 'mid', 'closing']) {
        const entry = schedule.find(s => s.day_of_week === d && s.shift_type === shift)
        byDay[d][shift] = entry?.employee_name || '(unassigned)'
      }
    }

    return {
      weekStart,
      scheduleByDay: Object.entries(byDay).map(([day, shifts]) => ({
        day: DAYS_OF_WEEK[Number(day)],
        opening: shifts.opening,
        mid: shifts.mid,
        closing: shifts.closing,
      })),
      totalAssigned: schedule.filter(s => s.employee_name).length,
      gaps: 21 - schedule.filter(s => s.employee_name).length,
    },
  })

export const getEmployeeScheduleTool = tool({
  description: 'Get a specific employee\'s shifts for the week, formatted as a personal schedule summary.',
  inputSchema: z.object({
    employeeName: z.string().describe('The employee name'),
    weekStart: z.string().describe('Week start date (Sunday) in YYYY-MM-DD format'),
  }),
  execute: async ({ employeeName, weekStart }) => {
    const schedule = await getWeekSchedule(weekStart)
    const shifts = schedule
      .filter(s => s.employee_name === employeeName)
      .map(s => ({
        day: DAYS_OF_WEEK[s.day_of_week],
        shift: s.shift_type,
      }))

    return {
      employee: employeeName,
      weekStart,
      shifts,
      totalShifts: shifts.length,
    }
  })
})
```

```

    }
  },
})

```

Create `lib/agents/communication-agent.ts`:

```

// lib/agents/communication-agent.ts

import { ToolLoopAgent, InferAgentUIMessage } from 'ai'
import { formatWeekSummaryTool, getEmployeeScheduleTool } from '../tools/communication-tools'

export const communicationAgent = new ToolLoopAgent({
  model: 'anthropic/claude-sonnet-4.5',
  instructions: `You are the Communication Agent for Fabulosa Books.

Your job is to create clear, friendly, human-readable schedule communications. You draft messages that could be posted in the breakroom, sent in a group text, or shared in a team chat.

Tone guidelines:
- Warm and collegial – this is a small, close-knit team
- Use first names
- Be concise but complete
- For individual schedules, mention any notable details (Saturday shifts, closing shifts)
- For weekly summaries, format as a clean, readable table

Example tone: "Hey team! Here's next week's schedule. Marcus, you've got Saturday closing (2-9pm) – thanks for taking that one. Let Alvin know ASAP if anything doesn't work."`,
  tools: {
    formatWeekSummary: formatWeekSummaryTool,
    getEmployeeSchedule: getEmployeeScheduleTool,
  },
})

export type CommunicationAgentUIMessage = InferAgentUIMessage<typeof communicationAgent>

```

Step 51: Add the Communication Agent to the coordinator.

Update `lib/agents/coordinator-agent.ts` to include the new subagent:

```

// Add to the imports
import { communicationAgent } from './communication-agent'

// Add this tool alongside the other subagent tools
const draftCommunicationSubagent = tool({
  description: 'Ask the Communication Agent to draft a human-readable schedule summary, notification, or message for the team. Use this when the user wants a formatted schedule to share.',
  inputSchema: z.object({
    question: z.string().describe('What to draft or format'),
  }),
  execute: async ({ question }) => {
    const result = await communicationAgent.generate({
      messages: [{ role: 'user', content: question }],
    })
    return { answer: result.text }
  },
})

// Add to the coordinator's tools:
// tools: {
//   ...existing tools,

```

```
// draftCommunication: draftCommunicationSubagent,  
// },
```

Update the coordinator's instructions to mention the Communication Agent:

```
You coordinate scheduling by delegating to four specialist agents:  
1. **Availability Agent** – checks who is free and when  
2. **Fairness Agent** – ensures equitable shift distribution  
3. **Coverage Agent** – validates store staffing requirements  
4. **Communication Agent** – drafts human-readable schedule summaries and notifications
```

[IMAGE: Updated architecture diagram with four specialist agents instead of three, now including the Communication Agent with a speech bubble icon. Caption: "The full team: four specialists, one coordinator."]

Step 52: Test the Communication Agent.

Try:

- **"Draft the team schedule for this week"** — The coordinator calls the Communication Agent, which formats a clean, readable summary.
- **"What's Billy's schedule this week?"** — A personal schedule summary.
- **"Write a message telling the team about next week's schedule"** — A friendly notification draft.

[IMAGE: Screenshot showing the chat panel with a nicely formatted weekly schedule summary, complete with a friendly message in the Communication Agent's warm tone]

Error Handling and Guardrails

Step 53: Handle impossible schedules gracefully.

What happens when the agents can't find a valid schedule? Maybe half the team is on vacation. Maybe there's an event requiring extra staff. The coordinator should explain *what* it couldn't solve and *why*, rather than failing silently.

Add to the coordinator's instructions:

```
When you cannot create a complete schedule:  
- Explain which constraints conflict (e.g., "Only 4 regular staff are available but we need 3 shifts x 7 days")  
- Fill what you can and clearly mark remaining gaps  
- Suggest solutions: "Consider asking Johnny Ray or Carly for extra on-call shifts"  
- Never silently leave shifts unassigned – always flag gaps explicitly
```

[IMAGE: Screenshot showing the coordinator gracefully handling a difficult week — explaining "I was able to fill 18 of 21 shifts. The remaining 3 gaps are..." with specific suggestions for resolution]

Cost and Performance

Step 54: Understand the cost implications.

Each agent call uses an AI model. In a multi-agent system, the coordinator might call 3-4 subagents per request, each of which might make 2-3 tool calls. That's real usage.

Practical guidance:

- The coordinator needs the most capable model since it orchestrates complex reasoning. `anthropic/claude-sonnet-4` is a good default.
- For simpler agents (Coverage Agent, which mostly validates rules), you could use a faster, cheaper model like `anthropic/claude-haiku-4-5`.
- The AI Gateway tracks usage across all providers. Check your dashboard to monitor costs.

To use a cheaper model for a specific agent, just change the model string:

```
// In coverage-agent.ts – this agent's job is simple enough for Haiku
export const coverageAgent = new ToolLoopAgent({
  model: 'anthropic/claude-haiku-4.5',
  // ... rest stays the same
})
```

[IMAGE: Screenshot of the Vercel AI Gateway usage dashboard showing model usage breakdown by agent. Caption: “The AI Gateway dashboard lets you monitor cost and usage per model.”]

Deploy the Final Version

Step 55: Push the complete system.

```
git add -A
git commit -m "Complete multi-agent system: add Communication Agent, error handling, cost optimization"
git push
```

Vercel deploys automatically. Your complete multi-agent scheduling system is live.

[IMAGE: Screenshot of the final deployed application — the full layout with schedule grid, chat panel showing agent activity, and the Vercel URL in the browser. Caption: “The finished product: a multi-agent scheduling system, live on the internet.”]

What You’ve Learned

Let’s recap what this tutorial covered:

Concept	What You Learned	Where
AI Agents	Programs with goals and tools that reason toward solutions	Part 2
Tools	Specific capabilities (database queries, validation) given to agents	Parts 5-6
The Tool Loop	Think → use tool → observe → think again → respond	Part 5
Specialization	Breaking complex problems into focused agents	Part 6
Orchestration	A coordinator agent that delegates to specialists via tool calls	Part 6

Concept	What You Learned	Where
Subagent Pattern	Wrapping agents as tools for other agents to call	Part 6
Streaming UI	Real-time agent activity in the browser	Parts 5-6
Type Safety	InferAgentUIMessage for typed tool results in React	Part 5
Graceful Degradation	Handling impossible constraints transparently	Part 7
Cost Management	Using appropriate models for different agent complexity levels	Part 7

What's Next

Ideas for extending the system:

- **Employee self-service:** An agent that handles shift swap requests between employees
- **Calendar integration:** Connect to Google Calendar so the agent knows about personal conflicts
- **Event scheduling:** A specialist agent for staffing readings, signings, and community events
- **Authentication:** Role-based access so only Alvin and Melissa can trigger full reschedules
- **Notifications:** Connect the Communication Agent to email or SMS

The patterns you've learned—tools, agents, orchestration, subagents—apply to any domain. Customer support systems, content pipelines, data analysis workflows, DevOps automation. Anywhere you have a complex task that benefits from specialized reasoning coordinated by a central intelligence, multi-agent orchestration is the architecture.

For Fabulosa Books, the paper schedule on the breakroom wall can finally retire. Not just because the schedule is digital now—but because the *thinking* is automated. Alvin can focus on curating books, hosting events, and running the store he loves. The agents handle the rest.

Built with Next.js, Vercel AI SDK, Neon, and Vercel. Inspired by the real Fabulosa Books in San Francisco's Castro neighborhood.